

Asistente Turístico Inteligente

Sistema Inteligente de Turismo Inclusivo, Seguro y Adaptativo para Tarija

Campo	Detalle
Estudiante	Sebastian Ramos Justiniano
Materia	Inteligencia Artificial
Actividad	Actividad 13 — IA Sprint Challenge, Impacto Social
Paradigma IA	RAG + Reglas Simbólicas (IA Híbrida)
ODS	ODS 10 — Reducción de las Desigualdades
GitHub	github.com/SEBS-RJ/tarijaapp

1. Justificación Social y Objetivo SMART

En la ciudad de Tarija, turistas nacionales e internacionales enfrentan dificultades para acceder a información confiable sobre rutas, transporte, seguridad y lugares de interés. La información disponible se encuentra dispersa en redes sociales o recomendaciones informales, generando desigualdad en el acceso a servicios turísticos digitales. Este problema afecta especialmente a adultos mayores, turistas extranjeros y personas con bajos recursos tecnológicos.

El proyecto se alinea con el **ODS 10 (Reducción de las Desigualdades)** al democratizar el acceso a información turística personalizada, segura y gratuita mediante IA.

Objetivo SMART

Criterio	Descripción
Específico	Desarrollar un asistente turístico de IA para Tarija que responda consultas en lenguaje natural sobre lugares de interés.
Medible	Sistema con cobertura $\geq 90\%$ en consultas turísticas y tiempo de respuesta $< 100\text{ms}$.
Alcanzable	Prototipo funcional con RAG sobre base de conocimiento local, implementado en Python sin dependencias externas.
Relevante	Responde a la necesidad real de orientación turística en Tarija, ciudad con creciente flujo de visitantes.
Temporal	Prototipo funcional completado en 3 sprints de 24 horas cada uno.

2. Bitácora Ágil — 3 Sprints

Sprint 1: Descubrimiento y Modelado

Actividades realizadas:

- Análisis del problema turístico de Tarija y justificación ODS 10.
- Modelado CommonKADS: modelos de organización, tareas, agentes, conocimiento y comunicación.
- Diseño de arquitectura híbrida RAG + Reglas Simbólicas.
- Definición de los 6 agentes del sistema y sus responsabilidades.
- Creación del prompt maestro del sistema y la estructura de carpetas del proyecto.

Entregable Sprint 1: Documento CommonKADS + Diagrama de arquitectura + 6 archivos de agentes .md con objetivos, reglas, entradas, salidas y consideraciones éticas.

Sprint 2: Construcción del núcleo de IA

Actividades realizadas:

- Construcción de la base de conocimiento turístico de Tarija (3 archivos .txt: lugares, gastronomía/transporte/seguridad, cultura/eventos).
- Implementación del motor RAG en Python puro: DocumentLoader, TFIDFRetriever, SymbolicRules, ResponseGenerator.
- Motor TF-IDF sin dependencias externas: indexación de 31 chunks, 640 términos únicos.
- 9 reglas simbólicas IF-THEN: seguridad nocturna, presupuesto bajo, tiempo limitado, turista extranjero, emergencias.
- 12 pruebas de inferencia ejecutadas con métricas documentadas.

Entregable Sprint 2: rag_engine.py funcional + test_inferencia.py con métricas: 91.7% cobertura, 0.4ms promedio, Nivel Avanzado en rúbrica.

Sprint 3: Integración, Ética y Pulido

Actividades realizadas:

- Desarrollo de la interfaz conversacional con Streamlit (app.py).
- Panel de contexto del usuario: hora, presupuesto, tiempo disponible, turista extranjero.
- Métricas de sesión en tiempo real: consultas totales, confianza promedio.
- Quick replies (consultas rápidas) para facilitar la interacción.
- Auditoría de sesgos y análisis ético del sistema.
- Documentación técnica final y generación de este PDF.

Entregable Sprint 3: app.py Streamlit funcional + PDF de entrega completo.

3. Descripción Técnica — Arquitectura

Paradigma elegido: RAG + Reglas Simbólicas (IA Híbrida)

¿Por qué RAG y no solo un LLM? Un LLM puro (ej. GPT-4) no conoce datos locales específicos de Tarija: horarios de lugares, precios en bolivianos, zonas de seguridad, rutas de micros. Sin RAG, el modelo alucinaría información incorrecta. Con RAG, el sistema primero recupera conocimiento verificado de la base local y luego genera la respuesta fundamentada en esos datos reales.

¿Por qué Reglas Simbólicas además del RAG? Ciertas decisiones turísticas requieren razonamiento explícito y transparente: si es de noche y el usuario pregunta por rutas, el sistema DEBE advertir sobre seguridad. Si el presupuesto es menor a 50 BOB, DEBE filtrar opciones gratuitas. Las reglas IF-THEN garantizan comportamiento predecible y explicable, especialmente en contextos de seguridad.

Flujo de datos del sistema

Consulta usuario → Tokenización TF-IDF → Búsqueda vectorial en chunks → Top-K recuperados → Motor de reglas simbólicas → Generador de respuesta → Respuesta final con alertas y fuentes

Stack tecnológico

Componente	Tecnología	Justificación
Motor RAG	Python puro (TF-IDF)	Sin dependencias de pago; portable a Colab
Reglas simbólicas	Python (IF-THEN)	Razonamiento explicable y transparente
Interfaz usuario	Streamlit	Despliegue rápido, demo visual para el docente
Base conocimiento	Archivos .txt	Portable, auditable, actualizable sin BD
Tokenización	Python re + Counter	Sin NLTK; funciona sin conexión internet
Métricas	Python nativo	Cobertura, confianza, tiempo de respuesta

Reglas simbólicas implementadas

Condición (IF)	Acción (THEN)
hora >= 22 OR hora < 6 + query de ruta	Alerta seguridad nocturna + recomendar taxi
presupuesto < 50 BOB	Filtrar lugares gratuitos + recomendar micros
tiempo_disponible < 120 min	Itinerario corto, lugares céntricos
extranjero = True	Incluir contexto cultural + requisitos de ID
query contiene 'perdido/robo/emergencia'	Mostrar 110/119/165 como prioridad máxima

4. Evidencia de Funcionamiento

Métricas del Sprint 2 (test_inferencia.py)

Métrica	Resultado	Nivel rúbrica
Pruebas ejecutadas	12	—
Cobertura (con resultado)	91.7%	Nivel Avanzado ■
Sin resultado	1 / 12	Emergencia sin keyword
Confianza promedio	0.1514	—
Tiempo promedio respuesta	0.4 ms	—
Chunks indexados	31	—
Términos únicos	640	—
Dependencias externas	Ninguna	—

Repositorio y código fuente

Repositorio GitHub: github.com/SEBS-RJ/tarijaapp

Archivos principales:

- src/rag_engine.py — Motor RAG completo (TFIDFRetriever, SymbolicRules, ResponseGenerator)
- src/test_inferencia.py — 12 pruebas con métricas
- src/app.py — Interfaz Streamlit
- data/knowledge/*.txt — Base de conocimiento turístico de Tarija
- agents/*.md — 6 agentes documentados (CommonKADS)

Ejecución local

```
# Instalar dependencias pip install streamlit # Correr pruebas Sprint 2 python  
src/test_inferencia.py # Levantar interfaz Streamlit (Sprint 3) streamlit run  
src/app.py
```

5. Análisis de Impacto y Ética

Impacto social esperado

- Democratiza el acceso a información turística de Tarija para cualquier visitante.
- Reduce la dependencia de contactos locales para orientarse en la ciudad.
- Promueve pequeños negocios tarijeños (bodegas artesanales, mercados locales).
- Mejora la seguridad de turistas con alertas contextuales en tiempo real.

Auditoría de sesgos y riesgos éticos

Riesgo identificado	Impacto potencial	Medida de mitigación
Información desactualizada	Recomendación incorrecta	Indicar fecha en cada chunk; proceso de actualización mensual
Sesgo hacia lugares conocidos	Baja visibilidad de negocios pequeños	Incluir explícitamente mercados, bodegas artesanales y barrios periféricos
Dependencia tecnológica	Exclusión digital	Sistema funciona sin internet (base local); interfaz simple
Falsa sensación de seguridad	Riesgo para el turista	Alertas siempre conservadoras; recomendar verificar con fuentes locales
Privacidad de ubicación	Exposición de datos	No se almacena ni comparte ubicación del usuario; sesión temporal

Limitaciones del modelo

- TF-IDF es sensible a sinónimos (ej. 'vino' vs 'enología'): mejorable con embeddings semánticos.
- La base de conocimiento es estática: requiere actualización manual periódica.
- No hay personalización por historial de usuario (MVP sin persistencia).
- Las reglas simbólicas son deterministas; no aprenden de interacciones previas.

Mejoras propuestas para versión futura

- Reemplazar TF-IDF por embeddings semánticos (sentence-transformers) para mayor precisión.
- Conectar base de conocimiento a Supabase para actualizaciones en tiempo real.
- Integrar LLM (via API) para generación de respuestas más naturales.
- Agregar módulo de feedback del usuario para aprendizaje continuo.